

enam-skills-assessment — engineering.py session

Exported 2026-06-06 06:20 | 79 turns

Claude 2026-06-05 12:05

Let me check the memory index and then read the relevant files.

You 2026-06-05 12:05

[Request interrupted by user]

You 2026-06-05 13:45

in the engineering.py file, Write Python equity feature-engineering functions using:
pandas, numpy, ta (ema_indicator, sma_indicator, rsi, average_true_range),
pandas_ta (supertrend), scipy.stats.percentileofscore.

<helpers>

```
calculate_percentile(series: np.ndarray) -> float
percentileofscore(series, series[-1], kind='strict')
```

```
rolling_percentile(price_change: pd.Series, window: int) -> pd.Series
rolling(window).apply(calculate_percentile, raw=True)
```

```
calc_streak(price_change: pd.Series) -> np.ndarray
+N = N consecutive up days, -N = N consecutive down days, 0 = flat
Use np.select + groupby(~mask.cumsum()).cumsum() pattern for both directions
```

```
compute_atr(group: pd.DataFrame, window: int = 14) -> pd.Series
Return all-NaN series if len < window, else ta average_true_range
```

```
rolling_linreg_1d(y: np.ndarray, window: int, return_slope: bool = False) -> np.ndarray
No loops — use np.convolve with k_y=ones(window), k_xy=arange(window)[::-1]
Compute slope via (sum_xy - window*x_mean*y_mean) / x_var
y_hat_last = intercept + slope*(window-1)
Prepend (window-1) NaNs; if return_slope=True return slope instead of y_hat_last
```

```
sharpe_momentum(sharpe: np.ndarray, freq: str, window: int) -> np.ndarray
freq mapping → d:(n=252,factor=10) | w:(n=52,factor=5) | m:(n=12,factor=2)
lr_line = rolling_linreg_1d(sharpe, n)
roc = concat([NaN*(window-1), lr_line[window-1:] - lr_line[:len-window+1]])
return factor * roc
```

</helpers>

<main>

```
compute_features_d(price_df: pd.DataFrame) -> pd.DataFrame
Input cols: open, high, low, close, deltpct
```

Add in order:

1. Lagged OHLC b1-b6: {open/high/low/close}_b{i} = .shift(i)
 2. price_change = close.pct_change()
price_change_across_b50/b252 = pct_change(50/252)
 3. sharpe_252: 252-day rolling annualised Sharpe ($\mu/\sigma * \sqrt{252}$), zero-sigma → NaN
sharpe_252_sma3: sma_indicator(sharpe_252, 3)
 4. sharpe_mmt = sharpe_momentum(sharpe_252.to_numpy(float), freq='d', window=10)
 5. Supertrend: pta.supertrend(high,low,close, length=10, multiplier=3)
- Take first 2 cols, rename by splitting on '_' and lowercasing → [supert, supertd]

```
If result is None or empty → assign NaN to both cols
6. Rolling highs: 52w_high(252), 2yr_high(504), 3yr_high(756), min_periods=2
7. EMAs: ema10, ema21 | SMAs: sma50, sma100, sma200
Delivery %: delpct_sma10/50/100/200
8. atr_14 = compute_atr(data, 14)
net_change_atr = (close - close_b1) / atr_14, inf→NaN, round 4dp
9. rsi_7, rsi_7_sma3, rsi_14, rsi_14_sma3
10. crsi = (rsi(close,3) + rsi(Series(calc_streak(price_change)),2)
+ rolling_percentile(price_change,100)) / 3

Return enriched DataFrame.
</main>
```

Claude 2026-06-05 13:45

Let me check the current state of the engineering.py file first.

Claude 2026-06-05 13:47

The file is written. The two IDE warnings on lines 23/28 (`groupby` on bool) are false positives — the static type checker infers `bool` from the comparison but at runtime these are `pd.Series[bool]`, so `.groupby().cumsum()` is valid pandas. The broad `except` on line 128 is intentional since `pta.supertrend` can raise various error types.

Here's a quick summary of what's in the file:

```
| Helper | What it does |
|---|---|
| `calculate_percentile` | `percentileofscore` of last element, strict kind |
| `rolling_percentile` | Rolling window apply of the above |
| `calc_streak` | Consecutive up/down day counter using `groupby(~mask.cumsum()).cumsum()` |
| `compute_atr` | Returns all-NaN if `len < window`, else `ta.average_true_range` |
| `rolling_linreg_1d` | Vectorised rolling OLS via `np.convolve` — no Python loops |
| `sharpe_momentum` | ROC of the linreg fit-line, scaled by freq factor |

`compute_features_d` adds the 10 feature groups in order: lagged OHLC → price changes →
Sharpe/momentum →
Supertrend → rolling highs → EMA/SMA/delivery% → ATR → RSI variants → ConnorsRSI.
```

You 2026-06-05 14:16

write a python function that takes the price df along with computed daily features using the compute_features_d function and adds:

1. Weekly 30 SMA
2. Weekly supertrend indicator
3. Weekly absolute gains
4. Weekly percentage gains
5. Weekly ATR
6. Weekly Net Change ATR
7. Weekly 7 per rsi
8. SMA of Weekly 7 per rsi
9. Weekly volume
10. Monthly absolute
11. Monthly percentage change
12. Monthly ATR
13. Monthly Net change ATR
14. Monthly 7 per rsi

15. SMA of Monthly 7 per rsi
16. Monthly volume

along each daily entry in the daily features df. In case there are similar indicators between daily / weekly / monthly add an _d, _w, _m for indentification of timeframe.

Write separate functions for generating weekly features and monthly features. These functions should take the

features_df as the only input argument and return the dataframe with added features.

Claude 2026-06-05 14:17

Let me read the current engineering.py to append cleanly.

Claude 2026-06-05 14:24

Done. Here's a summary of what was added:

****Private helpers:****

- `\_resample\_ohlcv` — resamples OHLCV (+ volume if present) to any pandas freq via `agg`
- `\_merge\_to\_daily` — `merge\_asof(..., direction='backward')` to pin each daily row to the most recent completed week/month
- `\_supertrend\_cols` — shared supertrend logic for any timeframe suffix; gracefully NaN-fills on failure

****Public functions:****

| Function | Adds columns |

|---|---|

| `add\_weekly\_features(features\_df)` | `sma30\_w`, `supert\_w`, `supertd\_w`, `abs\_gain\_w`, `pct\_gain\_w`, `atr\_14\_w`, `net\_change\_atr\_w`, `rsi\_7\_w`, `rsi\_7\_sma3\_w`, `volume\_w`* |

| `add\_monthly\_features(features\_df)` | `abs\_gain\_m`, `pct\_gain\_m`, `atr\_14\_m`, `net\_change\_atr\_m`, `rsi\_7\_m`,

`rsi\_7\_sma3\_m`, `volume\_m`* |

| `compute\_features\_dwm(price\_df)` | Runs all three, then renames shared daily cols: `atr\_14`→`atr\_14\_d`, `net\_change\_atr`→`net\_change\_atr\_d`, `rsi\_7`→`rsi\_7\_d`, `rsi\_7\_sma3`→`rsi\_7\_sma3\_d`, `supert`→`supert\_d`, `supertd`→`supertd\_d` |

*`volume\_w` / `volume\_m` only appear if a `volume` column exists in the input. Weekly resamples on `W-FRI`

(Indian market week-end); monthly on `ME` (month-end).

You 2026-06-05 14:34

I tried running main.py but got below error:

Traceback (most recent call last):

File "C:\Users\Kushagra Saxena\Documents\enam-skills-assessment\src\main.py", line 14, in <module>

features_df = compute_features_dwm(price_df)

^^

File "C:\Users\Kushagra Saxena\Documents\enam-skills-assessment\src\engineering.py", line 299, in compute_features_dwm

df = add_weekly_features(df)

^^

File "C:\Users\Kushagra Saxena\Documents\enam-skills-assessment\src\engineering.py", line 229, in add_weekly_features

w = _resample_ohlcv(features_df, _WEEKLY_FREQ)

^^

File "C:\Users\Kushagra Saxena\Documents\enam-skills-assessment\src\engineering.py", line 193, in _resample_ohlcv

```
.resample(freq)
^^^^^^^^^^^^^^^^
File "C:\Users\Kushagra Saxena\anaconda3\envs\markets\Lib\site-packages\pandas\core\generic.py", line
9790, in
resample
return get_resampler(
^^^^^^^^^^^^^^^^
File "C:\Users\Kushagra Saxena\anaconda3\envs\markets\Lib\site-packages\pandas\core\resample.py", line
2050,
in get_resampler
return tg._get_resampler(obj, kind=kind)
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "C:\Users\Kushagra Saxena\anaconda3\envs\markets\Lib\site-packages\pandas\core\resample.py", line
2272,
in _get_resampler
raise TypeError(
TypeError: Only valid with DatetimeIndex, TimedeltaIndex or PeriodIndex, but got an instance of
'RangeIndex'
```

Claude 2026-06-05 14:34

Let me check main.py and data_manager.py to understand how the data is structured.

Claude 2026-06-05 14:34

Let me check the CSV structure to see the date column name.

Claude 2026-06-05 14:35

The issue is two-fold: `load_ohlcv_data` returns a `RangeIndex` (date stays as a column), and `_resample_ohlc` / `_merge_to_daily` both assume a `DatetimeIndex`. I'll fix both helpers to detect and use a `'date'` column when the index isn't datetime.

Claude 2026-06-05 14:37

Two targeted changes:

****`\_resample\_ohlcv`**** — before calling `.resample()`, if the index isn't a `DatetimeIndex`, it looks for a `'date'` column and sets it as the index on a copy. The original `features df` is unmodified.

```

**`_merge_to_daily`** — instead of always doing `reset_index(names='_date_')` (which yields integers
when the
index is a `RangeIndex`), it now:

```

- Uses the DatetimeIndex path when available (original behaviour)
- Falls back to parsing a ``date`` column as the merge key when the index is a ``RangeIndex``
- Drops the temporary ``_date_`` column and restores a clean ``RangeIndex`` on return, so the output matches the input's shape

You 2026-06-05 15:28

The values from `add_weekly_features` and `add_monthly_features` aren't coming out correct. For ex. the weekly volume has to be the rolling sum of daily volume. Monthly volume has to be the rolling sum of monthly volume.

Similarly the absolute gains have to be the rolling sum of daily gains for the week or the month. The weekly or monthly gains at a date would be the weekly / monthly gains divided by the closing price of last day or previous week for weekly compute and last day of previous month for monthly compute. Once we get these base

features right, other features should come out correct. Please fix this.

Claude 2026-06-05 15:29

The current approach resamples to calendar bars and uses `.diff()` / `.pct_change()` on those bars, which is wrong. The correct approach: volume and gains are **rolling sums on daily data** (5-day window for weekly, 21-day for monthly); ATR, RSI, SMA, and Supertrend stay on resampled OHLC bars and get merged back. Let me read the current weekly/monthly functions.

You 2026-06-05 15:30

[Request interrupted by user for tool use]

You 2026-06-05 15:31

Do not use blanket 5 day and 21 day rolling periods for weekly or monthly. Judge the start and end of the week or the month by the dates.

Claude 2026-06-05 15:34

Let me read the current state of the functions to edit precisely.

Claude 2026-06-05 15:34

Now I have the full picture. The fix is:

- **Volume**: `groupby(Grouper(freq=...))` cumsum` within each actual calendar week/month
- **Abs gain / pct gain**: reference close = last close of the *previous* completed week/month, found via `merge_asof`` on a derived "prev-period-end" date per row
- **ATR, RSI, SMA, Supertrend**: stay on resampled bars (correct), `net_change_atr` uses the calendar-correct `abs_gain`

Claude 2026-06-05 15:34

The exact string didn't match. Let me read the precise file content.

Claude 2026-06-05 15:36

The IDE warning on line 295 (`dayofweek`` on `DatetimeIndex``) is a false positive — `pd.DatetimeIndex.dayofweek`` is valid and works at runtime.

Here's what changed and why each piece is now correct:

**`_prev_period_ref_close``** — shared helper that, given a set of `lookup_dates`` (one per row), finds the last

available close on or before each lookup date via `merge_asof(..., direction='backward')``. This handles holidays/gaps gracefully.

Weekly gains:

- `prev_friday = monday_of_week - 3 days`` — this is *always* the Friday of the previous calendar week, regardless of day-of-week. For Monday through Friday of week W it maps identically to the same prior Friday,

so all five days share the same reference close.

- `abs_gain_w = close - ref_close_w`` (price move since last Friday's close)

- `pct_gain_w = abs_gain_w / ref_close_w``

****Monthly gains:****

- ``prev_month_end = first_day_of_month - 1 day`` → last calendar day of the prior month, then ``merge_asof`` picks up the last actual trading close on or before that date.

****Volume:****

- ``groupby(pd.Grouper(freq='W-FRI'))['volume'].cumsum()`` — accumulates volume day by day within each Mon-Fri

calendar week, resetting at the start of each new week. Same pattern for monthly with ``ME``.

****ATR, RSI, SMA, Supertrend**** — still computed on resampled OHLC bars (the correct domain), then merged back

to daily rows via ``_merge_to_daily``.

You 2026-06-05 15:46

The data is not coming right. For ex. if you look at features.csv in the features dir inside data dir and look at weekly volume of AMBER symbol on 2026-05-29 it should be 68065 since the week that ended on 29th may started on 22nd may and had 4 trading days. the daily volume on those 4 days were 18171, 10576, 15421, and 23897 respectively. When you are grouping data, also take the symbol into the grouping consideration as the data has multiple tickers.

Claude 2026-06-05 15:48

Two bugs: the cumsum and reference-close lookups don't group by symbol, and ``_merge_to_daily`` doesn't restore the original index (so per-symbol concat breaks). Let me read the full relevant section first.

Claude 2026-06-05 15:48

Now I have everything I need. Three changes:

1. ****`\_merge\_to\_daily`****: stamp ``_orig_pos_`` before sorting, restore the caller's exact index at the end
2. ****`add\_weekly\_features` / `add\_monthly\_features`****: renamed to ``_single`` helpers; public wrappers dispatch per-symbol and concat

Claude 2026-06-05 15:51

Here's what changed and why each fix is correct:

****`\_merge\_to\_daily`**** — adds ``_orig_pos_`` (0...n) before the sort, sorts back by it after the merge, then calls

``set_index(features_df.index)`` to restore whatever index the caller had (RangeIndex slice like [100...199] for

the ZEE group, DatetimeIndex, etc.). Previously it did ``reset_index(drop=True)`` which wiped the original positions.

****`\_add\_weekly/monthly\_features\_single`**** — identical to the old functions; just renamed to make clear they

expect a single-symbol slice. The ``dayofweek`` call is now on a ``pd.Series`` (via ``pd.Series(dates).dt.dayofweek``) instead of directly on the ``DatetimeIndex``, which also fixes the IDE false-positive.

****`\_apply\_per\_symbol`**** — generic dispatcher: if there's a ``symbol`` column with more than one ticker, it ``groupby('symbol', sort=False)`` to get each symbol's contiguous slice (with its original RangeIndex positions), applies the function, concatenates and ``sort_index()`` restores the original row order. Single-symbol or

no-symbol data skips the overhead.

`**`add_weekly_features` / `add_monthly_features`**` — now thin wrappers over ``_apply_per_symbol``.

You 2026-06-05 16:28

Write Python monthly and weekly equity feature functions.

Libraries: pandas, numpy, ta (ema_indicator, sma_indicator, rsi), pandas_ta as pta.

Reuse from prior session: sharpe_momentum, calc_streak, rolling_percentile, compute_atr.

NOTATION (use throughout):

LAGS_loop(col, 1..N) → for loop: `data[f'{col}_b{i}'] = data[col].shift(i)`

LAGS_dict(col, 1..N) → dict comprehension + `pd.concat` for `{col}_b1..bN`

FWD_dict(col, 1..N) → dict comprehension + `pd.concat` for `{col}_f1..fN` via `shift(-i)`

<new_helpers>

`wilder_ma(close: pd.Series, window: int) -> pd.Series`

`ewm(alpha=1/window, adjust=False).mean()`

`calc_drawdown_streak(*args) -> pd.Series` [9 positional args, logic unknown]

Stub: `return pd.Series(np.nan, index=args[0].index)`

</new_helpers>

SHARED BLOCK — identical in both functions, define inline:

`sharpe(w, ann): mu/sigma * sqrt(ann)`, zero-sigma→NaN via `np.divide(..., where=sig!=0)`

`supertrend: pta.supertrend(high,low,close, length=10, multiplier=3)`

→ first 2 cols, rename by `split('_')[0].lower()` → [supert, supertd]; if None/empty → NaN cols

`crsi: (rsi(close,3) + rsi(Series(calc_streak(price_change)),2) + rolling_percentile(price_change,100)) / 3`

`net_change_atr: ((close - close_b1) / atr_14).round(4), inf→NaN`

<monthly>

`compute_features_m(price_df: pd.DataFrame) -> pd.DataFrame`

Input cols: open, high, low, close

1. `LAGS_dict(open/high/low/close, 1..2)` → single concat

2. `high_24m = rolling(24, min_periods=12).max()`; `high_36m = rolling(36, min_periods=12).max()`

`high_24m_b1, high_36m_b1 = .shift(1)`

3. `price_change = close.pct_change()`

`LAGS_dict(price_change, 1..2)`

`price_change_across_b6/b12 = pct_change(6/12)`

`FWD_dict(price_change_across, f1,f2,f3,f6) [shift(-1,-2,-3,-6)]`

4. `sharpe` for `w` in [12,24,36], `ann=sqrt(12)` → `sharpe_12/24/36`

`sharpe_mmt = sharpe_momentum(sharpe_24.to_numpy(float), freq='d', window=10)`

5. `supertrend` block

6. `ema5, ema10`; `LAGS_dict(ema5, 1..2)`; `LAGS_dict(ema10, 1..2)`

7. `atr_14`; `net_change_atr` block; `LAGS_dict(net_change_atr, 1..4)`

8. `nr4: ([net_change_atr_b1,b2,b3].abs().max(axis=1) < 0.35)`

`| ([b1,b2,b3,b4].abs().max(axis=1) < 0.35)`

9. `rsi_7`; `rsi_7_sma3 = sma_indicator(rsi_7,3)`; `LAGS_dict(rsi_7_sma3, 1..2)`

10. `crsi` block; `LAGS_dict(crsi, 1..2)`

11. `price_struct = np.select on (high vs high_b1) x (low vs low_b1):`

`HH & HL→HHHL | HH & LL→HHLL | LH & HL→LHHL | LH & LL→LHLL`; default=NaN

`LAGS_dict(price_struct, 1..5)`

12. `drawdown_streak = calc_drawdown_streak(price_change, price_change_b1, price_change_b2,`

`high, high_b1, high_b2, price_struct, price_struct_b1, price_struct_b2)`

13. `rally_month = ((close >= high_24m_b1) & (price_change > 0)).astype(int)`

```
LAGS_dict(rally_month, 1..3); FWD_dict(rally_month, 1..3)
rally_in_next_3_prds = (rally_month_f1+f2+f3 > 0).astype(int)
label = ((rally_in_next_3_prds==1) | (price_change_across_f3>=0.2)).astype(int)
Return data.copy()
</monthly>
```

```
<weekly>
compute_features_w(price_df: pd.DataFrame) -> pd.DataFrame
Input cols: open, high, low, close
```

```
1. LAGS_loop(open/high/low/close, 1..6)
2. price_change = close.pct_change()
3. ema10, sma30, wdma30 = wilder_ma(close,30)
LAGS_loop(ema10/sma30/wdma30, 1..6)
4. sharpe w=104, ann=sqrt(52) → sharpe_104
sharpe_mmt = sharpe_momentum(sharpe_104.to_numpy(float), freq='d', window=10)
5. supertrend block; LAGS_loop(supert, 1..4); LAGS_loop(supertd, 1..4)
6. rsi_7; rsi_7_sma3 = sma_indicator(rsi_7,3); LAGS_loop(rsi_7_sma3, 1..2)
7. atr_14; net_change_atr block; LAGS_loop(net_change_atr, 1..4)
8. crsi block; LAGS_loop(crsi, 1..2)
</weekly>
```

Claude 2026-06-05 16:29

Let me read the full current file to see the existing `compute\_features\_w` and exactly where to place things.

Claude 2026-06-05 16:31

Now I'll replace the broken `compute\_features\_w` and add `wilder\_ma`, `calc\_drawdown\_streak`, and the new
`compute\_features\_m`.

Claude 2026-06-05 16:32

The old `compute\_features\_w` is gone — the file now ends at line 386. I'll append all new code at the end.

Claude 2026-06-05 16:37

All diagnostics are informational only — no errors. Here's a summary of what was written:

****New helpers:****

```
|||
|---|---|
| `wilder_ma(close, window)` | Wilder's smoothed MA via `ewm(alpha=1/window, adjust=False)` |
| `calc_drawdown_streak(*args)` | Stub returning NaN series — 9-arg signature reserved for full logic |
| `_st_block(data)` | Shared supertrend block used by both functions; adds `supert`/`supertd` in-place |
```

****`compute\_features\_w`**** — 8 blocks per spec: OHLC lags (loop b1..6) → `price\_change` → EMAs/SMA/WilderMA with lags → Sharpe(104, $\sqrt{52}$) + momentum → supertrend with lags b1..4 → RSI/sma3 with lags → ATR/net_change_atr with lags → ConnorsRSI with lags.

****`compute\_features\_m`**** — 13 blocks per spec: OHLC lags (dict b1..2) → rolling 24m/36m highs → price change family (lags, b6/b12 cross-period, f1/f2/f3/f6 forward) → Sharpe(12/24/36, $\sqrt{12}$) + momentum → supertrend → EMA5/10 with lags → ATR/net_change_atr with lags → NR4 flag → RSI/sma3 with lags → ConnorsRSI with lags

→

price structure (HHHL/HHLL/LHHL/LHLL) with lags b1..5 → drawdown streak stub → rally month + label.

Key design choices: ``price_change_across_f{i}` = `close.pct_change(i).shift(-i)`` (forward i-period return), which makes ``label`s `price_change_across_f3 >= 0.2`` read as "20%+ return over the next 3 months". ``LAGS_dict`` blocks use a single ``pd.concat`` per group to avoid repeated column assignment overhead.

You 2026-06-05 16:43

write `compute_features_dwm` to also call `compute_features_w` and `compute_features_m` per symbol. Also there is a `_st_block` func for weekly and monthly computes and a `_supertrend_cols` for daily compute. Can we have just one function for supertrend that can be used in daily, weekly, and monthly computes?

Claude 2026-06-05 16:43

Let me read the current file to see the exact state of all the supertrend functions and ``compute_features_dwm``.

Claude 2026-06-05 16:46

Now I have the full picture. Three targeted edits: (1) unify the two supertrend functions, (2) update all callers, (3) rewrite ``compute_features_dwm``.

Claude 2026-06-05 16:46

Now I have everything I need. I'll do three focused edits: replace both supertrend helpers with one, update all call sites, then rewrite ``compute_features_dwm``.

Claude 2026-06-05 16:47

The multi-timeframe infrastructure was removed during the file modification. Let me read the full current file.

Claude 2026-06-05 16:48

Now I have the complete picture. I'll make four targeted edits:

1. Replace ``_supertrend_cols` + `compute_features_dwm`` stub with ``_supertrend` + helpers + full `compute_features_dwm``
2. Remove ``_st_block``
3. Update the two ``_st_block`` call sites in ``compute_features_w`` and ``compute_features_m``
4. Update the inline supertrend block in ``compute_features_d``

You 2026-06-05 16:51

This session is being continued from a previous conversation that ran out of context. The summary below covers the earlier portion of the conversation.

Summary:

1. Primary Request and Intent:

The user is building an equity feature engineering pipeline in ``src/engineering.py`` for a multi-symbol stock market dataset. The overall goal is to produce a comprehensive feature DataFrame combining daily, weekly, and monthly technical indicators.

The conversation progressed through these explicit requests:

1. Write daily feature engineering helpers and ``compute_features_d``
2. Write ``add_weekly_features``, ``add_monthly_features``, and ``compute_features_dwm`` that merge

bar-level

weekly/monthly indicators onto daily rows

3. Fix `TypeError: Only valid with DatetimeIndex` when calling `.resample()` on a RangeIndex DataFrame
4. Fix incorrect weekly/monthly values: "Do not use blanket 5 day and 21 day rolling periods for weekly or monthly. Judge the start and end of the week or the month by the dates."
5. Fix multi-symbol grouping issue (weekly volume mixing symbols): "When you are grouping data, also take the symbol into the grouping consideration as the data has multiple tickers."
6. Write standalone `compute\_features\_w` and `compute\_features\_m` with detailed specs (wilder_ma, calc_drawdown_streak, LAGS_loop/dict/FWD_dict notation)
7. ****Most recent (pending):**** "write compute_features_dwm to also call compute_features_w and compute_features_m per symbol. Also there is a _st_block func for weekly and monthly computes and a _supertrend_cols for daily compute. Can we have just one function for supertrend that can be used in daily, weekly, and monthly computes?"

2. Key Technical Concepts:

- Multi-symbol equity OHLCV data (NSE India stocks: AMBER, DBL, NIFTYBEES, WELSPUN, ZEE)
- Technical indicators: EMA, SMA, Wilder's MA, ATR, RSI, Supertrend, ConnorsRSI, Sharpe ratio
- Rolling linear regression without loops using `np.convolve`
- Sharpe momentum (ROC of linreg fit-line on Sharpe series)
- `pandas\_ta.supertrend` for Supertrend indicator
- Multi-timeframe analysis: daily / weekly (W-FRI) / monthly (ME)
- `pd.merge\_asof(direction='backward')` for as-of merging bar-level indicators onto daily rows
- Per-symbol groupby processing to avoid cross-symbol contamination
- Calendar-based rolling: `groupby(pd.Grouper(freq='W-FRI')).cumsum()` for weekly volume
- `merge\_asof` with prev-period-end lookup dates for correct weekly/monthly gain reference prices
- Preserving original DataFrame index through `\_orig\_pos\_` tracking after `merge\_asof` reordering
- ConnorsRSI = (RSI(close,3) + RSI(streak_series,2) + rolling_percentile(price_change,100)) / 3
- Price structure classification: HHHH / HHLL / LHHL / LHLL via `np.select`

3. Files and Code Sections:

****`src/engineering.py`**** (441 lines — primary file, completely built from scratch)

Current structure:

```

Lines 1-7:  imports (ta, pandas_ta, pandas, numpy, scipy.stats.percentileofscore)
Lines 10-22: constants (_WEEKLY_FREQ='W-FRI', _MONTHLY_FREQ='ME', _DAILY_RENAMES dict)
Lines 25-95: helper functions
Lines 98-183: compute_features_d
Lines 186-198: _supertrend_cols (TO BE REPLACED)
Lines 201-204: compute_features_dwm (STUB — needs rewrite)
Lines 207-441: wilder_ma, calc_drawdown_streak, _st_block (TO BE REPLACED), compute_features_w,
compute_features_m
    
```

Key helpers currently in file:

```

def calculate_percentile(series: np.ndarray) -> float:
    return percentileofscore(series, series[-1], kind='strict')

def rolling_percentile(price_change: pd.Series, window: int) -> pd.Series:
    return price_change.rolling(window).apply(calculate_percentile, raw=True)

def calc_streak(price_change: pd.Series) -> np.ndarray:
    up = price_change > 0; down = price_change < 0
    up_streak = up.groupby((~up).cumsum()).cumsum().where(up, 0)
    
```

```

down_streak = down.groupby((~down).cumsum()).cumsum().where(down, 0)
return (up_streak - down_streak).to_numpy()

def compute_atr(group: pd.DataFrame, window: int = 14) -> pd.Series:
    if len(group) < window: return pd.Series(np.nan, index=group.index)
    return average_true_range(group['high'], group['low'], group['close'], window=window)

def rolling_linreg_1d(y: np.ndarray, window: int, return_slope: bool = False) -> np.ndarray:
    # vectorized rolling OLS via np.convolve – no loops
    # k_y=ones(window), k_xy=arange(window)[::-1]
    # returns y_hat_last (or slope if return_slope=True), prepended with (window-1) NaNs

def sharpe_momentum(sharpe: np.ndarray, freq: str, window: int) -> np.ndarray:
    freq_map = {'d': (252, 10), 'w': (52, 5), 'm': (12, 2)}
    # lr_line = rolling_linreg_1d(sharpe, n)
    # roc = concat([NaN*(window-1), lr_line[window-1:] - lr_line[:len-window+1]])
    # return factor * roc

def wilder_ma(close: pd.Series, window: int) -> pd.Series:
    return close.ewm(alpha=1 / window, adjust=False).mean()

def calc_drawdown_streak(price_change, price_change_b1, price_change_b2,
                        high, high_b1, high_b2,
                        price_struct, price_struct_b1, price_struct_b2) -> np.ndarray:
    # Uses np.select with 4 conditions, then calc_streak on result

```

Two supertrend functions that need to be unified:

```

# Currently at line 186 – returns suffixed 2-col DataFrame
def _supertrend_cols(high, low, close, suffix: str) -> pd.DataFrame:
    # returns DataFrame with columns ['supert_{suffix}', 'supertd_{suffix}']

# Currently at line 237 – modifies data in-place, returns data
def _st_block(data: pd.DataFrame) -> pd.DataFrame:
    # adds data['supert'] and data['supertd'] columns

```

`compute\_features\_w` (lines 253-312) — takes weekly OHLC bars:

- 1. LAGS_loop(open/high/low/close, b1..6)
- 2. price_change = close.pct_change()
- 3. ema10, sma30, wdma30=wilder_ma(close,30); LAGS_loop each b1..6
- 4. sharpe_104 with sqrt(52); sharpe_mmt via sharpe_momentum(freq='d', window=10)
- 5. `data = \_st\_block(data)` + lags supert/supertd b1..4
- 6. rsi_7, rsi_7_sma3; lags b1..2
- 7. atr_14, net_change_atr; lags b1..4
- 8. crsi; lags b1..2

`compute\_features\_m` (lines 315-441) — takes monthly OHLC bars:

- 1. LAGS_dict(open/high/low/close, b1..2) single concat
- 2. high_24m, high_36m, their b1 lags
- 3. price_change, lags b1..2, pct_change(6/12), forward f1/f2/f3/f6
- 4. sharpe_12/24/36 with sqrt(12); sharpe_mmt from sharpe_24
- 5. `data = \_st\_block(data)`
- 6. ema5, ema10; lags b1..2
- 7. atr_14, net_change_atr; lags b1..4
- 8. nr4 narrow-range flag
- 9. rsi_7, rsi_7_sma3; lags b1..2

- 10. crsi; lags b1..2
- 11. price_struct (HHHL/HHLL/LHHL/LHLL); lags b1..5
- 12. drawdown_streak via calc_drawdown_streak(9 args)
- 13. rally_month, lags b1..3, fwd f1..3, rally_in_next_3_prds, label

`compute\_features\_dwm` (lines 201-204) — CURRENTLY A STUB:

```
def compute_features_dwm(price_df: pd.DataFrame) -> pd.DataFrame:
    df = compute_features_d(price_df)
    return df
```

****MISSING from file**** (were present in earlier version, removed by user/linter during file modification):

- `\_resample\_ohlc(df, freq)` — resamples OHLCV to given frequency, handles RangeIndex with 'date' column
- `\_merge\_to\_daily(features\_df, indicator\_df)` — `merge\_asof` backward merge, preserves original index via `\_orig\_pos`
- `\_get\_dates(df)` — extracts DatetimeIndex from index or 'date' column
- `\_prev\_period\_ref\_close(close, dates, lookup\_dates, resample\_freq)` — finds last close before each period boundary
- `\_period\_cumsum(df, col, dates, grouper\_freq)` — cumulative sum within calendar periods
- `\_add\_weekly\_features\_single(features\_df)` — single-symbol weekly bar overlay
- `\_add\_monthly\_features\_single(features\_df)` — single-symbol monthly bar overlay
- `\_apply\_per\_symbol(features\_df, fn)` — per-symbol dispatcher with sort_index() restoration
- `add\_weekly\_features(features\_df)` — public wrapper
- `add\_monthly\_features(features\_df)` — public wrapper

****`src/data\_manager.py`****

```
def load_ohlc_data() -> pd.DataFrame:
    ohlc_df = pd.DataFrame()
    for file in os.listdir(OHLCV_DIR):
        ohlc_df = pd.concat([ohlc_df, pd.read_csv(OHLCV_DIR / file)], ignore_index=True)
    return ohlc_df
```

Returns RangeIndex DataFrame with columns: `(unnamed), date, symbol, sc\_code, sc\_group, open, high, low, close, prev\_close, volume, turnover`

****`src/main.py`****

```
price_df = load_ohlc_data()
features_df = compute_features_dwm(price_df)
features_df.to_csv(FEAT_DIR / "features_dwm.csv", index=False)
```

4. Errors and fixes:

- ****TypeError: Only valid with DatetimeIndex**** when calling `.resample()`:
- Cause: `load\_ohlc\_data()` uses `ignore\_index=True` → RangeIndex, 'date' stays as column
- Fix: `\_resample\_ohlc` checks `isinstance(work.index, pd.DatetimeIndex)`, falls back to `work.index = pd.to\_datetime(df['date'])`. `\_merge\_to\_daily` similarly checks for DatetimeIndex vs 'date' column.
- ****Wrong weekly/monthly values**** (volume mixing days, gains wrong):
- Initial approach used `.diff()` and `.pct\_change()` on resampled weekly bars, and fixed 5/21-day rolling for volume
- User feedback: "Do not use blanket 5 day and 21 day rolling periods for weekly or monthly. Judge the start and end of the week or the month by the dates."
- Fix: `\_prev\_period\_ref\_close` uses `merge\_asof` on `prev\_friday = monday\_of\_week - 3 days` to find the

last
close of the previous W-FRI week. `\_period\_cumsum` uses `groupby(pd.Grouper(freq='W-FRI')).cumsum()`
for
calendar-correct volume accumulation.

- **Multi-symbol contamination** in weekly volume/gains:
- User feedback: "When you are grouping data, also take the symbol into the grouping consideration as the data has multiple tickers." Example: AMBER 2026-05-29 volume = 68065 (sum of 18171+10576+15421+23897).
- Fix: Added `\_apply\_per\_symbol` dispatcher; renamed functions to `\_add\_weekly/monthly\_features\_single`; fixed
`\_merge\_to\_daily` to preserve original index using `\_orig\_pos\_` stamp + `set\_index(features\_df.index)` at end.

- **`wilder\_ma` unresolved reference** in old `compute\_features\_w`:
- The original file had `wilder\_ma` called but not defined.
- Fix: Added `wilder\_ma` definition before `compute\_features\_w`.

- **IDE error** (line 239 in old code):
- IDE flagged "Unresolved attribute reference 'str' for class 'list'"
- Fix: Changed to `[c.split('\_')[0].lower() for c in st\_cols.columns]` which avoids the `.str` accessor pattern.

- **IDE warning** on DatetimeIndex:
- Fix: Changed to `pd.Series(dates).dt.dayofweek.values` to access through a Series with `.dt` accessor.

5. Problem Solving:

- **Calendar-correct weekly reference close**: For any day D in week W (Mon-Fri ending on Friday), the reference close is the last close of the previous W-FRI period. Computed as: `prev\_friday = monday\_of\_week - 3 days`, then `merge\_asof(direction='backward')` on the weekly close series indexed by Friday dates. This handles holidays gracefully (picks the most recent trading close on or before the lookup date).

- **Multi-symbol per-group processing**: `features\_df.groupby('symbol', sort=False)` splits into per-symbol groups that retain their original RangeIndex positions. Each group processed independently.
`pd.concat(parts).sort\_index()` restores original row order.

- **Index preservation through merge_asof**: `merge\_asof` sorts by the merge key, destroying original row order. Fixed by stamping `\_orig\_pos\_ = np.arange(len(daily\_r))` before merge, sorting back by `\_orig\_pos\_` after, then `set\_index(features\_df.index)` to restore exact original index labels.

6. All user messages:

1. "in the engineering.py file, Write Python equity feature-engineering functions using: pandas, numpy, ta (ema_indicator, sma_indicator, rsi, average_true_range), pandas_ta (supertrend), scipy.stats.percentileofscore." [with detailed helper and compute_features_d specs]

2. "write a python function that takes the price df along with computed daily features using the compute_features_d function and adds: 1. Weekly 30 SMA, 2. Weekly supertrend indicator, 3. Weekly absolute gains, 4. Weekly percentage gains, 5. Weekly ATR, 6. Weekly Net Change ATR, 7. Weekly 7 per rsi, 8. SMA of Weekly 7 per rsi, 9. Weekly volume, 10. Monthly absolute, 11. Monthly percentage change, 12. Monthly ATR, 13. Monthly Net change ATR, 14. Monthly 7 per rsi, 15. SMA of Monthly 7 per rsi, 16. Monthly volume... In case there are similar indicators between daily / weekly / monthly add an _d, _w, _m for identification of timeframe. Write separate functions for generating weekly features and monthly features."

3. "I tried running main.py but got below error: TypeError: Only valid with DatetimeIndex, TimedeltaIndex or

PeriodIndex, but got an instance of 'RangeIndex'. Fix this."

4. "The values from `add_weekly_features` and `add_monthly_features` aren't coming out correct. For ex. the weekly volume has to be the rolling sum of daily volume. Monthly volume has to be the rolling sum of monthly volume.

Similarly the absolute gains have to be the rolling sum of daily gains for the week or the month. The weekly or monthly gains at a date would be the weekly / monthly gains divided by the closing price of last day or previous week for weekly compute and last day of previous month for monthly compute. Once we get these base features right, other features should come out correct. Please fix this."

5. [User rejected tool call] "Do not use blanket 5 day and 21 day rolling periods for weekly or monthly. Judge the start and end of the week or the month by the dates."

6. "The data is not coming right. For ex. if you look at `features.csv` in the `features` dir inside `data` dir and look at weekly volume of AMBER symbol on 2026-05-29 it should be 68065 since the week that ended on 29th may started on 22nd may and had 4 trading days. the daily volume on those 4 days were 18171, 10576, 15421, and 23897 respectively. When you are grouping data, also take the symbol into the grouping consideration as the data has multiple tickers."

7. "Write Python monthly and weekly equity feature functions. Libraries: `pandas`, `numpy`, `ta` (`ema_indicator`, `sma_indicator`, `rsi`), `pandas_ta` as `pta`. Reuse from prior session: `sharpe_momentum`, `calc_streak`, `rolling_percentile`, `compute_atr`." [with full spec for `compute_features_w` and `compute_features_m` including `LAGS_loop/dict/FWD_dict` notation, `wilder_ma`, `calc_drawdown_streak`]

8. "write `compute_features_dwm` to also call `compute_features_w` and `compute_features_m` per symbol. Also there is a `_st_block` func for weekly and monthly computes and a `_supertrend_cols` for daily compute. Can we have just one function for supertrend that can be used in daily, weekly, and monthly computes?"

7. Pending Tasks:

1. ****Unify supertrend functions****: Replace ``_supertrend_cols`` (line 186) and ``_st_block`` (line 237) with a single ``_supertrend(high, low, close, suffix=None)`` function that returns a 2-column DataFrame with columns ``supert`` / ``supertd`` (when `suffix=None`) or ``supert_{suffix}`` / ``supertd_{suffix}`` (when `suffix` provided).

2. ****Update all supertrend call sites****:

- In ``compute_features_d`` (lines 134-145): replace inline `try/except` with ``st_df = _supertrend(high, low, close); data['supert'] = st_df['supert'].values; data['supertd'] = st_df['supertd'].values``
- In ``compute_features_w`` (line 282): replace ``data = _st_block(data)`` with ``data = pd.concat([data, _supertrend(data['high'], data['low'], data['close'])], axis=1)``
- In ``compute_features_m`` (line 357): same replacement

3. ****Rewrite ``compute_features_dwm`` to call all three compute functions per symbol and merge results:**

- Needs re-addition of ``_resample_ohlcv`` and ``_merge_to_daily`` helpers (removed from file)
- Per-symbol: call ``compute_features_d`` → resample to weekly → call ``compute_features_w`` → add ``_w`` suffix to feature cols → merge back → resample to monthly → call ``compute_features_m`` → add ``_m`` suffix → merge back → apply ``_DAILY_RENAMES``

8. Current Work:

Immediately before this summary request, the assistant was about to execute the changes for:

1. Replacing `\_supertrend\_cols` (line 186-198) and `\_st\_block` (lines 237-250) with a unified `\_supertrend` function
2. Rewriting `compute\_features\_dwm` (lines 201-204, currently a stub)

The unified function design:

```
def _supertrend(high: pd.Series, low: pd.Series, close: pd.Series,
               suffix: str = None) -> pd.DataFrame:
    col_s = 'supert' if suffix is None else f'supert_{suffix}'
    col_d = 'supertd' if suffix is None else f'supertd_{suffix}'
    try:
        st = pta.supertrend(high, low, close, length=10, multiplier=3)
        if st is None or st.empty:
            raise ValueError
        cols = st.iloc[:, :2].copy()
        cols.columns = [col_s, col_d]
        return cols
    except Exception:
        return pd.DataFrame({col_s: np.nan, col_d: np.nan}, index=high.index)
```

The `compute\_features\_dwm` rewrite design:

```
def compute_features_dwm(price_df: pd.DataFrame) -> pd.DataFrame:
    def _process_single(grp: pd.DataFrame) -> pd.DataFrame:
        # Daily features
        df = compute_features_d(grp)
        # Weekly: resample → compute_features_w → suffix _w → merge
        w_bars = _resample_ohlcw(grp, _WEEKLY_FREQ)
        w_feats = compute_features_w(w_bars)
        w_feats = w_feats.drop(columns=[c for c in ['open', 'high', 'low', 'close', 'volume'] if c in
w_feats.columns])
        w_feats = w_feats.add_suffix('_w')
        new_w = [c for c in w_feats.columns if c not in df.columns]
        if new_w:
            df = _merge_to_daily(df, w_feats[new_w])
        # Monthly: resample → compute_features_m → suffix _m → merge
        m_bars = _resample_ohlcw(grp, _MONTHLY_FREQ)
        m_feats = compute_features_m(m_bars)
        m_feats = m_feats.drop(columns=[c for c in ['open', 'high', 'low', 'close', 'volume'] if c in
m_feats.columns])
        m_feats = m_feats.add_suffix('_m')
        new_m = [c for c in m_feats.columns if c not in df.columns]
        if new_m:
            df = _merge_to_daily(df, m_feats[new_m])
        rename = {k: v for k, v in _DAILY_RENAMES.items() if k in df.columns}
        return df.rename(columns=rename)

    if 'symbol' not in price_df.columns or price_df['symbol'].nunique() <= 1:
        return _process_single(price_df)
    parts = [_process_single(grp) for _, grp in price_df.groupby('symbol', sort=False)]
    return pd.concat(parts).sort_index()
```

This also requires re-adding `\_resample\_ohlcw` and `\_merge\_to\_daily` which were removed from the file.

9. Optional Next Step:

Based on the most recent user request: "write compute_features_dwm to also call compute_features_w and compute_features_m per symbol. Also there is a _st_block func for weekly and monthly computes and a _supertrend_cols for daily compute. Can we have just one function for supertrend that can be used in daily, weekly, and monthly computes?"

The next step is to make three edits to `src/engineering.py`:

1. Replace the block `\_supertrend\_cols` (lines 186-198) and `\_st\_block` (lines 237-250) with the single unified `\_supertrend(high, low, close, suffix=None)` function

2. Update the three call sites:

- `compute\_features\_d` lines 134-145: replace inline try/except
- `compute\_features\_w` line 282: replace `data = \_st\_block(data)`
- `compute\_features\_m` line 357: replace `data = \_st\_block(data)`

3. Rewrite `compute\_features\_dwm` (lines 201-204) with per-symbol logic, re-adding `\_resample\_ohlcw` and `\_merge\_to\_daily` helpers since they were removed from the file

If you need specific details from before compaction (like exact code snippets, error messages, or content you

generated), read the full transcript at: C:\Users\Kushagra Saxena\.claude\projects\C--Users-Kushagra-Saxena-

Documents-enam-skills-assessment\7cd1235d-79e3-4f1b-99fc-8211295569e5.jsonl

Continue the conversation from where it left off without asking the user any further questions. Resume directly — do not acknowledge the summary, do not recap what was happening, do not preface with "I'll continue" or similar. Pick up the last task as if the break never happened.

Claude 2026-06-05 16:52

Good, the first edit is done. Now let me read the current state of the file to do the remaining edits accurately.

Claude 2026-06-05 16:52

Now I have a clear picture of the file. Let me make all remaining edits:

Claude 2026-06-05 16:52

Now remove `\_st\_block` and update the two call sites:

Claude 2026-06-05 16:53

Good, now fix the two call sites. Let me find their exact lines:

Claude 2026-06-05 16:53

Now verify no remaining `\_st\_block` or `\_supertrend\_cols` references and check the `\_supertrend` call site in

`compute\_features\_d` resolves correctly:

Claude 2026-06-05 16:53

All four call sites use `\_supertrend`. No remaining `\_st\_block` or `\_supertrend\_cols` references. Let me do a final sanity check on the `compute\_features\_dwm` function:

Claude 2026-06-05 16:54

Everything looks correct. Here's what was done:

****`\_supertrend\_cols` + `\_st\_block` → unified `\_supertrend`****

- Single function at line 178. Returns a 2-col DataFrame with `supert`/`supertd` (or `supert\_{suffix}`/`supertd\_{suffix}` when suffix is provided).

****Three call sites updated:****

- ``compute_features_d`` (line 135): ``st_df = _supertrend(high, low, close)`` — assigns values directly
- ``compute_features_w`` (line 317): ``pd.concat([data, _supertrend(...)]`` — appends columns
- ``compute_features_m`` (line 392): same pattern

****`compute\_features\_dwm` rewritten (lines 237-254):****

- Per-symbol via ``groupby('symbol')``
- Calls ``compute_features_d`` → resamples to W-FRI → calls ``compute_features_w``, strips base OHLC, adds ``_w`` suffix, backward-merges to daily → same for ``_MONTHLY_FREQ`` / ``compute_features_m`` / ``_m``
- Re-added ``_resample_ohlc`` (lines 197-206) and ``_merge_bar_to_daily`` (lines 209-234) helpers

You 2026-06-05 18:56

use the `get_swing_nodes` function inside `compute_features_m` to get columns of rolling 'peak' price and 'trough' price, use `def val order_h = 12` and `order_l = 8`. Do not attempt to add any old function that was previously there but now is deleted. I have purposely deleted certain functions that were previously generated which were not getting used anywhere.

Claude 2026-06-05 18:56

Let me read the current file to understand the context before making changes.

Claude 2026-06-05 18:57

``get_swing_nodes`` is already in the file (line 26). I'll add its usage inside ``compute_features_m`` right after the rolling highs section, since swing peaks/troughs are structurally similar price-level indicators. The columns are forward-filled so every row carries the most recent confirmed swing value.

Claude 2026-06-05 18:58

Done. Here's what was added to ``compute_features_m`` (after the rolling highs block):

```
peak_idx, trough_idx = get_swing_nodes(data, order_h=12, order_l=8)
data['peak'] = np.nan
data['trough'] = np.nan
if len(peak_idx[0]):
    data.iloc[peak_idx[0], data.columns.get_loc('peak')] = data['high'].iloc[peak_idx[0]].values
if len(trough_idx[0]):
    data.iloc[trough_idx[0], data.columns.get_loc('trough')] = data['low'].iloc[trough_idx[0]].values
data['peak'] = data['peak'].ffill()
data['trough'] = data['trough'].ffill()
```

- ``peak`` uses ``high`` at ``argrextrema(np.greater_equal, order=12)`` positions, then forward-fills — so every monthly bar carries the most recent confirmed swing high price
- ``trough`` uses ``low`` at ``argrextrema(np.less, order=8)`` positions, forward-filled similarly
- The ``len(...)`` guards handle the edge case where a symbol has too few bars to detect any extrema

You 2026-06-05 20:06

The `price_w` and `price_m` dataframes are created with the date as the first trading day of that week or the month. Write a python functions in `engineering.py` to map on the daily features df:

1. `'sl_b1'` across the relevant week if the day is anyday but 'Friday'. If it is a Friday, map `'sl'` from `features_w` for that particular ticker
2. `'peak_b1'` and `'trough_b1'` across the relevant month in `features_d`

Claude 2026-06-05 20:09


```

line
2000, in __init__
_OrderedMerge.__init__(
File "C:\Users\Kushagra Saxena\anaconda3\envs\markets\Lib\site-packages\pandas\core\reshape\merge.py",
line
1912, in __init__
_MergeOperation.__init__(
File "C:\Users\Kushagra Saxena\anaconda3\envs\markets\Lib\site-packages\pandas\core\reshape\merge.py",
line
802, in __init__
self._maybe_require_matching_dtypes(self.left_join_keys, self.right_join_keys)
File "C:\Users\Kushagra Saxena\anaconda3\envs\markets\Lib\site-packages\pandas\core\reshape\merge.py",
line
2125, in _maybe_require_matching_dtypes
_check_dtype_match(lk, rk, i)
File "C:\Users\Kushagra Saxena\anaconda3\envs\markets\Lib\site-packages\pandas\core\reshape\merge.py",
line
2121, in _check_dtype_match
raise MergeError(msg)
pandas.errors.MergeError: incompatible merge keys [0] dtype('<M8[ns]') and dtype('int64'), must be the
same
type

```

Claude 2026-06-05 20:18

The error is a dtype mismatch: ``features_w`` has a `RangeIndex (int64)` instead of a `DatetimeIndex`, so resetting the index produces an integer ``_bar_date`` column. The fix is to handle both cases — `DatetimeIndex` or a ``date`` column — and force both keys to ``datetime64`` before merging.

Claude 2026-06-05 20:18

The root cause was that `bar_df.reset_index()` was resetting a RangeIndex to integers instead of dates. The fix:

- If ``bar_df`` has a `**DatetimeIndex**`: reset and rename the first column to ``_bar_date_`` (original path)
- If ``bar_df`` has a `**'date'`` column: rename it to ``_bar_date_`` without touching the index
- Explicitly cast ``_bar_date_`` to ``datetime64`` in both branches before the merge, ensuring type parity with the daily ``date`` column

You 2026-06-05 20:19

Now getting below error:

Traceback (most recent call last):

File "C:\Users\Kushagra Saxena\Documents\enam-skills-assessment\main.py", line 25, in <module>
features d = map weekly sl to daily(features d, features w)

~~~~~

File "C:\Users\Kushagra Saxena\Documents\enam-skills-assessment\src\engineering.py", line 505, in  
map weekly sl to daily

```
result = asof join bars(features d, features w[w cols])
```

[illegible]

File "C:\Users\Kushagra Saxena\Documents\enam-skills-assessment\src\engineering.py", line 476, in asof join bars

```
raise ValueError("bar df must have a DatetimeIndex or a 'date' column")
```

```
ValueError: bar df must have a DatetimeIndex or a 'date' column
```

**Claude 2026-06-05 20:19**

Let me read `main.py` and the utils file to understand how `features\_w` is being constructed.

**Claude 2026-06-05 20:20**

The problem is two-fold:

1. `features\_w[w\_cols]` strips the `date` column before it reaches `\_asof\_join\_bars`
2. `features\_w` is multi-symbol, so the join needs to be done per symbol to avoid cross-symbol contamination

I'll fix both in the two mapping functions — explicitly set `date` as the DatetimeIndex before joining, and split by symbol.

**Claude 2026-06-05 20:20**

Two fixes applied:

1. `date` column stripped before join — instead of passing `features_w[feat_cols]` (which drops `date`), each symbol's bar DataFrame now has `date` set as its DatetimeIndex via `grp_w.set_index(pd.to_datetime(grp_w['date']))[feat_cols]` before entering `_asof_join_bars`
2. `Multi-symbol contamination` — both functions now `groupby('symbol')` and join each symbol's daily rows against only that symbol's weekly/monthly bars, then `pd.concat(...).sort_index()` to restore original order

**You 2026-06-05 21:26**

Like `map_monthly_swings_to_daily`, create a function to map `max_buy_price_b1` to `features_d` for the relevant dates.

**Claude 2026-06-05 21:26**

The pattern is identical to `map_monthly_swings_to_daily` — just a different column. I'll add it right after that function.

**Claude 2026-06-05 21:27**

`map_monthly_max_buy_to_daily` added, following the exact same per-symbol asof-join pattern. Call it in `main.py` as:

```
features_d = map_monthly_max_buy_to_daily(features_d, features_m)
```

The IDE note about "duplicated code fragment" is expected — the three monthly mapping functions are intentionally parallel by design.

**You 2026-06-05 22:16**

Like `map_weekly_sl_to_daily` function, map `supert` from `features_w` to `features_d` if the day is a Friday else map `supert_b1` for the relevant dates of the week. In fact if possible, merge with `map_weekly_sl_to_daily` function as logic is same. Rename the function to something more general, refactor the code accordingly.

**Claude 2026-06-05 22:16**

Let me read the current state of the mapping functions.

**Claude 2026-06-05 22:17**

Now I'll replace `map_weekly_sl_to_daily` with a general `map_weekly_to_daily` that accepts a list of base column names and applies the Friday/non-Friday logic to each.



**Claude 2026-06-06 06:17**

Good, I have enough to build the export. Let me write the script now.

**Claude 2026-06-06 06:18**

The em dash in the title is outside latin-1. I'll add a Unicode TTF font so the full chat content works too.

**Claude 2026-06-06 06:19**

DejaVu fonts are available via matplotlib. I'll update the script to use them.

**Claude 2026-06-06 06:19**

Now replace all `"Helvetica"` and `"Courier"` font calls with the Unicode variants: